Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования «Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Программирование графических процессоров

Расчетно-графическое задание

Выполнил: Студент 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич

Проверил преподаватель:
Малков Евгений Александрович

Новосибирск, 2024.

Оглавление

Постановка задачи 3

Описание архитектуры CUDA и «сырого» кода CUDA C 4

Описание библиотеки Thrust 5

Описание библиотеки cuBLAS 6

Описание OpenGL 7

Описание реализации 8

Результаты работы программы 10

Вывод: 12

Листинг программы 13

OpenGL 13

main.cu 18

utility.h 33

matrix_handler.h 22

cuda_handler.h 23

Постановка задачи

Задание: Сравнение производительности программ на основе интерфейса CUDA (пункт 1) и на основе OpenGL - вычислительных шейдеров (оценка “отлично”).

Цель: провести сравнительный анализ производительности программ, реализующих алгоритмы линейной алгебры с использованием библиотек Thrust, cuBLAS, "сырого" CUDA C кода и на основе OpenGL(вычислительные шейдеры). Для этого необходимо рассмотреть следующие задачи:

1. Изучить основные принципы работы библиотек Thrust и cuBLAS, а также "сырого" CUDA C кода.
2. Реализовать несколько алгоритмов линейной алгебры с использованием каждой из библиотек и "сырого" CUDA C кода.
3. Провести тестирование разработанных программ на различных объемах данных и сравнить полученные результаты в терминах времени выполнения операций линейной алгебры.
4. Проанализировать полученные результаты и сделать выводы о производительности каждого подхода и эффективности использования библиотек Thrust, cuBLAS, "сырым" CUDA C кодом и OpenGL.

Описание архитектуры CUDA и «сырого» кода CUDA C

CUDA (Compute Unified Device Architecture) — это архитектура параллельных вычислений, разработанная компанией NVIDIA для использования графических процессоров (GPU) в качестве вычислительных устройств. Она предоставляет возможность использовать вычислительные возможности GPU для выполнения параллельных вычислений.

CUDA позволяет писать приложения на языке программирования C или C++, которые могут выполняться на графическом процессоре. Это делает возможным ускорение вычислений в таких областях, как научные исследования, машинное обучение, анализ больших данных и другие.

"Сырой" код CUDA — это прямой доступ к возможностям графического процессора через использование языка CUDA C. Он позволяет написать свои собственные ядра (kernel), которые могут выполняться на GPU. Код ядер CUDA C напоминает обычный C-код, но включает дополнительные функции, такие как спецификаторы качества, которые указывают, как ядро должно выполняться на GPU.

"Сырой" код CUDA предоставляет более гибкий и точный контроль над выполнением кода на GPU. Он позволяет оптимизировать свой код для специфических характеристик архитектуры GPU, что может привести к ускорению вычислений и более эффективному использованию ресурсов GPU. Однако написание "сырого" кода CUDA может потребовать большего времени и усилий, чем использование библиотек, таких как cuBLAS и Thrust.

Описание библиотеки Thrust

Thrust — библиотека для обобщенного программирования на языке C++ с использованием графических процессоров NVIDIA CUDA. Библиотека предоставляет множество алгоритмов, контейнеров и функций, которые упрощают написание эффективных параллельных программ на CUDA C.

Thrust предоставляет широкий спектр функций для работы с массивами данных, включая сортировку, свертку, поиск, перемешивание и т.д. Она также предоставляет контейнеры, такие как векторы, матрицы и многомерные массивы, которые могут быть использованы для хранения и манипулирования данными. Кроме того, Thrust поддерживает различные операции над контейнерами, такие как слияние, преобразование, фильтрацию и другие.

Одной из главных преимуществ Thrust является ее интеграция с языком C++, что позволяет использовать знакомый синтаксис языка и библиотеки STL для работы с данными на GPU. Это позволяет легко переносить существующий код на CUDA на платформу, поддерживаемую Thrust.

Thrust также предоставляет удобный интерфейс для работы с различными типами данных, включая пользовательские типы, что позволяет использовать ее в широком диапазоне задач вычислительной математики, машинного обучения и других областях.

В целом, Thrust является мощным инструментом для разработки эффективных параллельных программ на CUDA C, позволяя сосредоточиться на решении задач вычислительной математики и машинного обучения, а не на деталях параллельного программирования на GPU.

Описание библиотеки cuBLAS

cuBLAS — библиотека линейной алгебры для GPU, разработанная компанией NVIDIA и оптимизированная для использования с графическими процессорами NVIDIA CUDA. Библиотека предоставляет реализации на GPU стандартных алгоритмов линейной алгебры, таких как умножение матриц, решение систем линейных уравнений, нахождение собственных значений и векторов, QR-разложение и другие.

cuBLAS имеет широкий функционал для работы с различными типами матриц и векторов, в том числе с плотными и разреженными матрицами. Библиотека обеспечивает максимальную производительность вычислений на GPU, используя оптимизированные алгоритмы и обходя ограничения, которые могут возникнуть при использовании центрального процессора (CPU).

Одним из главных преимуществ cuBLAS является ее оптимизация под специфичные характеристики архитектуры GPU, такие как высокая параллельность и высокая пропускная способность. Библиотека позволяет получить существенное ускорение вычислений линейной алгебры, что делает ее незаменимым инструментом для разработки многих приложений в области научных исследований, машинного обучения и других областей.

В целом, cuBLAS является мощным инструментом для разработки эффективных приложений на CUDA, обеспечивающим максимальную производительность и оптимизацию для работы с матрицами и векторами на GPU.

Описание OpenGL

Основным принципом работы OpenGL является получение наборов векторных графических примитивов в виде точек, линий и треугольников с последующей математической обработкой полученных данных и построением растровой картинки на экране и/или в памяти. Векторные трансформации и растеризация выполняются графическим конвейером (graphics pipeline), который по сути представляет собой дискретный автомат. Абсолютное большинство команд OpenGL попадает в одну из двух групп: либо они добавляют графические примитивы на вход в конвейер, либо конфигурируют конвейер на различное исполнение трансформаций.

OpenGL является низкоуровневым процедурным API, что вынуждает программиста диктовать точную последовательность шагов, чтобы построить результирующую растровую графику (императивный подход). Это является основным отличием от дескрипторных подходов, когда вся сцена передается в виде структуры данных (чаще всего дерева), которое обрабатывается и строится на экране. С одной стороны, императивный подход требует от программиста глубокого знания законов трёхмерной графики и математических моделей, с другой стороны — даёт свободу внедрения различных инноваций.

На базовом уровне OpenGL — это просто *спецификация*, то есть документ, описывающий набор функций и их точное поведение. Производители оборудования на основе этой спецификации создают *реализации* — библиотеки функций, соответствующих набору функций спецификации. Реализация призвана эффективно использовать возможности оборудования.

Описание реализации

Для провеления сравнения производительности было написанно две прораммы, которые представляют собой реализацию операций над матрицами, таких как транспонирование, умножение и скалярное произведение.

1. использует библиотеки thrust, cuBLAS и CUDA;
2. OpenGL.

Код программ представлен в листинге, ниже приведено краткое описание реализации.

Для проверки правильности транспонирования и умножения, программа включает функции transpose_check и multiply_check, которые сравнивают результаты выполнения соответствующих операций с эталонными значениями.

Для транспонирования матриц используются следующие функции:

- transpose_kernel, который выполняет транспонирование на GPU с помощью CUDA ядра.
- kernel_transpose, который также использует CUDA ядро, но в отличие от предыдущей функции, он возвращает время выполнения операции.
- thrust_transpose, который использует библиотеку thrust для выполнения транспонирования на GPU.
- cublas_transpose, который использует библиотеку cuBLAS для выполнения транспонирования на GPU.

Для умножения матриц используются следующие функции:

- matrix_multiply_kernel, который выполняет умножение матриц на GPU с помощью CUDA ядра.
- kernel_matrix_multiply, который также использует CUDA ядро, но в отличие от предыдущей функции, он возвращает время выполнения операции.
- thrust_matrix_multiply, который использует библиотеку thrust для выполнения умножения матриц на GPU.
- cublas_matrix_multiply, который использует библиотеку cuBLAS для выполнения умножения матриц на GPU.

Для скалярного произведения используются следующие функции:

- scalar_multiply_kernel, который выполняет скалярное произведение на GPU с помощью CUDA ядра.
- kernel_scalar_multiply, который также использует CUDA ядро, но в отличие от предыдущей функции, он возвращает время выполнения операции.
- thrust_scalar_multiply, который использует библиотеку thrust для выполнения скалярного произведения на GPU.
- cublas_scalar_multiply, который использует библиотеку cuBLAS для выполнения скалярного произведения на GPU.

OpenGL:

const char* matrixMulShaderSource – инициализация шейдера для умножения.

const char* matrixTransposeShaderSource – инициализация шейдера для транспонирования.

const char* matrixScalarMulShaderSource – инициализация шейдера для скалярного умножения.

void checkShaderError(GLuint shader) – функция проверки ошибки компиляции шейдера.

void checkProgramError(GLuint program) – функция проверки ошибки линковки.

В функции main() происходит инициализация матриц для всех операций, создание и заполнение буферов, запуск вычислительного шейдера.

Результаты работы программы

results.csv ×

РГЗ > OPENGL > results.csv

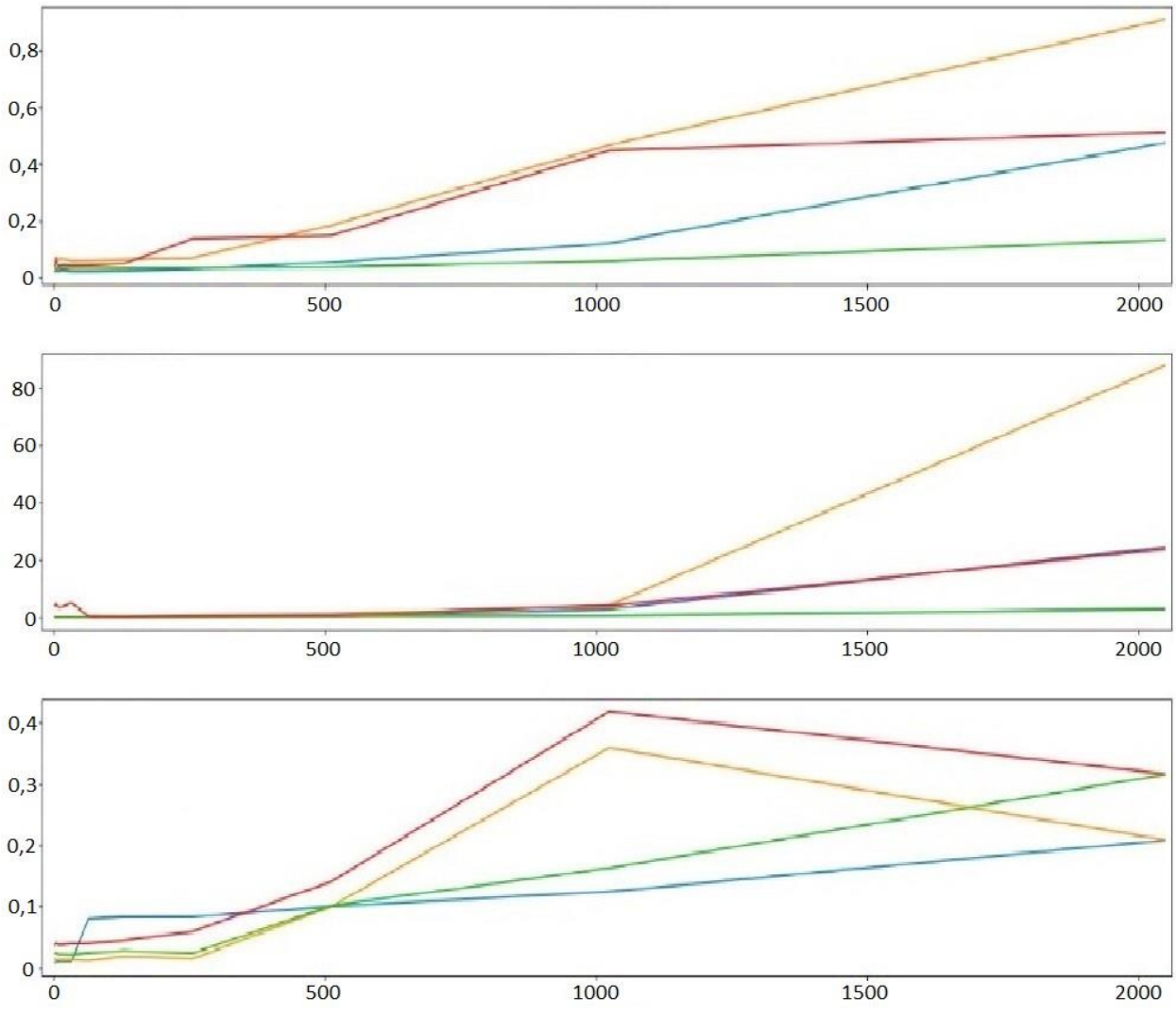
```
Matrix size (n^2); MT (OPENGL); MM (OPENGL); SM (OPENGL);
2;0.042321;4.04238;0.038033;
4;0.0614334;4.68421;0.039966;
8;0.039887;3.38927;0.038063;
16;0.043744;3.55522;0.038183;
32;0.042171;4.94601;0.039466;
64;0.042902;0.493533;0.039756;
128;0.047761;0.358456;0.044776;
256;0.137443;0.676884;0.059604;
512;0.147782;1.04094;0.14137;
1024;0.448417;3.92298;0.419542;
2048;0.511898;23.668;0.317557;
2;6.76539;15.518;6.32713;
4;1.77659;0.676498;0.577739;
8;6.38398;0.977379;0.777979;
16;7.80241;0.763171;0.955099;
32;0.680129;4.51156;0.256672;
64;0.15987;0.726504;0.122993;
128;14.2337;5.49781;4.07532;
256;0.940572;16.4308;1.35935;
512;0.782867;32.2079;0.660921;
1024;2.18028;205.687;1.34126;
2048;7.11383;1646.64;3.94283;
2;0.777908;7.58153;0.47327;
4;0.22235;0.678993;0.166605;
8;0.190844;0.591325;0.168071;
16;1.6847;2.43704;0.752132;
32;0.211033;0.601384;0.174708;
64;0.38867;1.97073;1.05012;
128;0.276973;0.975796;0.196221;
256;0.573086;3.60644;0.312947;
512;0.477874;23.5096;0.317558;
1024;1.11724;131.631;0.965098;
2048;3.28176;748.8;2.27734;

```

Рис. 1 – Результаты работы программы операций над матрицами OPENGL.

```
2;0.023552;0.061792;0.036096;0.01152;0.018848;0.292608;0.00768;0.014144;0.024096;
4;0.0256;0.070688;0.033568;0.01344;0.019328;0.278496;0.008672;0.013728;0.02432;
8;0.023328;0.062496;0.032992;0.011936;0.018752;0.288224;0.008864;0.013024;0.022208;
16;0.026624;0.065088;0.033216;0.012096;0.018976;0.281696;0.00912;0.013792;0.022144;
32;0.022688;0.059104;0.033312;0.011872;0.018112;0.280096;0.009216;0.013088;0.021728;
64;0.022528;0.059616;0.033216;0.013344;0.02208;0.277696;0.08064;0.01168;0.024384;
128;0.024576;0.06256;0.035008;0.023584;0.032832;0.388864;0.083552;0.018688;0.028352;
256;0.030336;0.0696;0.03408;0.056768;0.162784;0.378816;0.083712;0.015136;0.024;
512;0.053312;0.184384;0.036768;0.358144;0.552256;0.432704;0.100768;0.100544;0.102272;
1024;0.120448;0.467424;0.05792;2.72666;4.2;0.673536;0.124832;0.360352;0.1632;
2048;0.475776;0.910688;0.132672;24.5239;87.7206;2.84381;0.208672;0.209472;0.316448;
```

Рис. 2 – Результаты работы программы операций над матрицами CUDA/Thrust/cuBLAS.



---OpenGL ---Thrust ---cuBLAS ---CUDA

Рис. 3 – Графики зависимости времени выполнения операций (мс) от размерности матриц 1 - Транспонирование, 2 – Умножение, 3 – скалярное произведение.

Вывод:

1. Я изучил основные принципы работы библиотек Thrust и cuBLAS, а также "сырого" CUDA C кода.
2. Реализовал несколько алгоритмов линейной алгебры с использованием каждой из библиотек и "сырого" CUDA C кода.
3. Провел тестирование разработанных программ на различных объемах данных и сравнил полученные результаты.
4. На основе полученных данных были построены графики зависимости времени выполнения операций (мс) от размерности матриц.

Результаты показывают, что использование cuBLAS демонстрирует заметное повышение скорости выполнения операций с матрицами на GPU по сравнению с ручными реализациями на CUDA, OpenGL и Thrust. При этом, при сравнении "сырого" кодом с OpenGL, CUDA выигрывает благодаря использованию собственных оптимизаций, специфичных для видеокарты.

Листинг программы

OpenGL

```
#include <iostream>
#include <vector>
#include <chrono>
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <fstream>

using namespace std;

const char* matrixMulShaderSource = R"(
#version 430
layout (local_size_x = 16, local_size_y = 16) in;
layout (std430, binding = 0) buffer A {
    float a[];
};
layout (std430, binding = 1) buffer B {
    float b[];
};
layout (std430, binding = 2) buffer C {
    float c[];
};
uniform int size;
void main() {
    ivec2 gid = ivec2(gl_GlobalInvocationID.xy);
    if (gid.x < size && gid.y < size) {
        float sum = 0.0;
        for (int k = 0; k < size; k++) {
            sum += a[gid.y * size + k] * b[k * size + gid.x];
        }
        c[gid.y * size + gid.x] = sum;
    }
}
)";


const char* matrixTransposeShaderSource = R"(
#version 430
layout (local_size_x = 16, local_size_y = 16) in;
layout (std430, binding = 0) buffer A {
    float a[];
};
layout (std430, binding = 1) buffer B {
    float b[];
};
uniform int size;
void main() {
```

```
    ivec2 gid = ivec2(gl_GlobalInvocationID.xy);
    if (gid.x < size && gid.y < size) {
        b[gid.y * size + gid.x] = a[gid.x * size + gid.y];
    }
}
)";


const char* matrixScalarMulShaderSource = R"(
#version 430
layout (local_size_x = 16, local_size_y = 16) in;
layout (std430, binding = 0) buffer A {
    float a[];
};
layout (std430, binding = 1) buffer B {
    float b[];
};
uniform float scalar;
uniform int size;
void main() {
    ivec2 gid = ivec2(gl_GlobalInvocationID.xy);
    if (gid.x < size && gid.y < size) {
        b[gid.y * size + gid.x] = a[gid.y * size + gid.x] * scalar;
    }
}
)";

void checkShaderError(GLuint shader) {
    GLint success;
    glGetShaderiv(shader, GL_COMPILE_STATUS, &success);
    if (!success) {
        char infoLog[512];
        glGetShaderInfoLog(shader, 512, nullptr, infoLog);
        std::cerr << "Shader compilation failed:\n" << infoLog << std::endl;
    }
}

void checkProgramError(GLuint program) {
    GLint success;
    glGetProgramiv(program, GL_LINK_STATUS, &success);
    if (!success) {
        char infoLog[512];
        glGetProgramInfoLog(program, 512, nullptr, infoLog);
       std::cerr << "Shader program linking failed:\n" << infoLog << std::endl;
    }
}

int main(int argc, char** argv) {

    if (argc < 2) {
```

```
    return -1;
}

int matrixSize = atoi(argv[1]);

// Инициализация GLFW и GLEW
if (!glfwInit()) {
    std::cerr << "GLFW initialization failed" << std::endl;
    return -1;
}
glfwWindowHint(GLFW_VISIBLE, GLFW_FALSE);
GLFWwindow* window = glfwCreateWindow(1, 1, "", nullptr, nullptr);
if (!window) {
    std::cerr << "GLFW window creation failed" << std::endl;
    glfwTerminate();
    return -1;
}
glfwMakeContextCurrent(window);
if (glewInit() != GLEW_OK) {
    std::cerr << "GLEW initialization failed" << std::endl;
    glfwTerminate();
    return -1;
}

// Создание шейдерной программы
GLuint shader = glCreateShader(GL_COMPUTE_SHADER);
glShaderSource(shader, 1, &matrixMulShaderSource, nullptr);
glCompileShader(shader);
checkShaderError(shader);
GLuint program = glCreateProgram();
glAttachShader(program, shader);
glLinkProgram(program);
checkProgramError(program);
glUseProgram(program);
glDeleteShader(shader);

// Размер матрицы и количество экспериментов

// Инициализация матриц A и B с случайными значениями
std::vector<float> a(matrixSize * matrixSize), b(matrixSize * matrixSize);
for (int i = 0; i < matrixSize * matrixSize; i++) {
    a[i] = static_cast<float>(rand()) / RAND_MAX;
    b[i] = static_cast<float>(rand()) / RAND_MAX;
}

// Создание и заполнение буферов
GLuint buffers[3];
glGenBuffers(3, buffers);
glBindBufferBase(GL_SHADER_STORAGE_BUFFER, 0, buffers[0]);
```

```
            glBufferData(GL_SHADER_STORAGE_BUFFER,  matrixSize  *  matrixSize  *
sizeof(float), a.data(), GL_DYNAMIC_DRAW);
          glBindBufferBase(GL_SHADER_STORAGE_BUFFER, 1, buffers[1]);
            glBufferData(GL_SHADER_STORAGE_BUFFER,  matrixSize  *  matrixSize  *
sizeof(float), b.data(), GL_DYNAMIC_DRAW);
          glBindBufferBase(GL_SHADER_STORAGE_BUFFER, 2, buffers[2]);
            glBufferData(GL_SHADER_STORAGE_BUFFER,  matrixSize  *  matrixSize  *
sizeof(float), nullptr, GL_DYNAMIC_DRAW);

          // Запуск вычислительного шейдера и измерение времени выполнения
          glUseProgram(program);
          glUniform1i(glGetUniformLocation(program, "size"), matrixSize);
          glMemoryBarrier(GL_SHADER_STORAGE_BARRIER_BIT);

                    std::chrono::high_resolution_clock::time_point      start      =
std::chrono::high_resolution_clock::now();

          glDispatchCompute((matrixSize + 15) / 16, (matrixSize + 15) / 16, 1);
          glMemoryBarrier(GL_SHADER_STORAGE_BARRIER_BIT);
          glFinish();

                     std::chrono::high_resolution_clock::time_point      end      =
std::chrono::high_resolution_clock::now();
          std::chrono::duration<double, std::milli> elapsed_matrix_multiplication =
end - start;

          glDeleteProgram(program);
          glDeleteBuffers(3, buffers);

          shader = glCreateShader(GL_COMPUTE_SHADER);
          glShaderSource(shader, 1, &matrixTransposeShaderSource, nullptr);
          glCompileShader(shader);
          checkShaderError(shader);
          program = glCreateProgram();
          glAttachShader(program, shader);
          glLinkProgram(program);
          checkProgramError(program);
          glUseProgram(program);
          glDeleteShader(shader);

          glGenBuffers(2, buffers);
          glBindBufferBase(GL_SHADER_STORAGE_BUFFER, 0, buffers[0]);
            glBufferData(GL_SHADER_STORAGE_BUFFER,  matrixSize  *  matrixSize  *
sizeof(float), a.data(), GL_DYNAMIC_DRAW);
          glBindBufferBase(GL_SHADER_STORAGE_BUFFER, 1, buffers[1]);
            glBufferData(GL_SHADER_STORAGE_BUFFER,  matrixSize  *  matrixSize  *
sizeof(float), nullptr, GL_DYNAMIC_DRAW);


          glUseProgram(program);
```

```
            glUniform1i(glGetUniformLocation(program, "size"), matrixSize);
            glMemoryBarrier(GL_SHADER_STORAGE_BARRIER_BIT);

            start = std::chrono::high_resolution_clock::now();
            glDispatchCompute((matrixSize + 15) / 16, (matrixSize + 15) / 16, 1);
            glMemoryBarrier(GL_SHADER_STORAGE_BARRIER_BIT);
            glFinish();
            end = std::chrono::high_resolution_clock::now();
            std::chrono::duration<double, std::milli> elapsed_matrix_tanspose = end -
start;

            glDeleteProgram(program);
            glDeleteBuffers(3, buffers);

            shader = glCreateShader(GL_COMPUTE_SHADER);
            glShaderSource(shader, 1, &matrixScalarMulShaderSource, nullptr);
            glCompileShader(shader);
            checkShaderError(shader);
            program = glCreateProgram();
            glAttachShader(program, shader);
            glLinkProgram(program);
            checkProgramError(program);
            glUseProgram(program);
            glDeleteShader(shader);

            glGenBuffers(2, buffers);
            glBindBufferBase(GL_SHADER_STORAGE_BUFFER, 0, buffers[0]);
               glBufferData(GL_SHADER_STORAGE_BUFFER,  matrixSize  *  matrixSize  *
sizeof(float), a.data(), GL_DYNAMIC_DRAW);
            glBindBufferBase(GL_SHADER_STORAGE_BUFFER, 1, buffers[1]);
               glBufferData(GL_SHADER_STORAGE_BUFFER,  matrixSize  *  matrixSize  *
sizeof(float), nullptr, GL_DYNAMIC_DRAW);

            const float scalar = 2.0f;

            glUseProgram(program);
            glUniform1f(glGetUniformLocation(program, "scalar"), scalar);
            glUniform1i(glGetUniformLocation(program, "size"), matrixSize);
            glMemoryBarrier(GL_SHADER_STORAGE_BARRIER_BIT);


            start = std::chrono::high_resolution_clock::now();
            glDispatchCompute((matrixSize + 15) / 16, (matrixSize + 15) / 16, 1);
            glMemoryBarrier(GL_SHADER_STORAGE_BARRIER_BIT);
            glFinish();
            end = std::chrono::high_resolution_clock::now();
             std::chrono::duration<double, std::milli> elapsed_matrix_scalar = end -
start;
            ofstream file;
            file.open("results.csv", ios::app);
```

```
            if (file.tellp() == streampos(0))
            {
                    file << "Matrix size (n^2); MT (OPENGL); MM (OPENGL); SM (OPENGL);"
<< "\n";
            }
            file
                << matrixSize << ";"
                << elapsed_matrix_tanspose.count() << ";"
                << elapsed_matrix_multiplication.count() << ";"
                << elapsed_matrix_scalar.count() << ";"
                << "\n";
            file.close();
            // Освобождение ресурсов и завершение работы
            glDeleteProgram(program);
            glDeleteBuffers(3, buffers);
            glfwTerminate();

            return 0;
        }
```

main.cu

```
        #include <iostream>
        #include <iomanip>
        #include <stdio.h>
        #include <stdlib.h>
        #include <cstdlib>
        #include <chrono>
        #include <thread>
        #include <fstream>
        #include "utility.h"
        #include "cuda_handler.h"
        #include "matrix_handler.h"

        using namespace std;

        int main()
        {
            float *input_matrix_1, *input_matrix_2, *output_matrix;
            input_matrix_1 = (float *)malloc(NUM_ELEMENTS * sizeof(float));
            input_matrix_2 = (float *)malloc(NUM_ELEMENTS * sizeof(float));
            output_matrix = (float *)malloc(NUM_ELEMENTS * sizeof(float));

            init_matrix(input_matrix_1);
            this_thread::sleep_for(chrono::seconds(1));
            init_matrix(input_matrix_2);
            init_zero_matrix(output_matrix);

            if (PRINT_MODE)
            {
```

```
            cout << "BASE MATRICES:" << endl;
            cout << "input_matrix_1:" << endl;
            print_matrix(input_matrix_1);
            cout << "input_matrix_2:" << endl;
            print_matrix(input_matrix_2);
            cout << "output_matrix:" << endl;
            print_matrix(output_matrix);
        }

            float    *device_input_matrix_1,    *device_input_matrix_2,
*device_output_matrix;
        cudaMalloc(&device_input_matrix_1, NUM_ELEMENTS * sizeof(float));
        cudaMalloc(&device_input_matrix_2, NUM_ELEMENTS * sizeof(float));
        cudaMalloc(&device_output_matrix, NUM_ELEMENTS * sizeof(float));
         cudaMemcpy(device_input_matrix_1,  input_matrix_1,  NUM_ELEMENTS  *
sizeof(float), cudaMemcpyHostToDevice);
         cudaMemcpy(device_input_matrix_2,  input_matrix_2,  NUM_ELEMENTS  *
sizeof(float), cudaMemcpyHostToDevice);

        cout << "KERNEL TRANSPOSING:" << endl;
        init_zero_matrix(output_matrix);
        float elapsed_time_kernel_transpose = 0.0f;
                                elapsed_time_kernel_transpose             =
kernel_transpose(device_input_matrix_1, device_output_matrix);
          cudaMemcpy(output_matrix,  device_output_matrix,  NUM_ELEMENTS  *
sizeof(float), cudaMemcpyDeviceToHost);
        transpose_check(input_matrix_1, output_matrix);
        cout << "Time: " << elapsed_time_kernel_transpose << " ms" << endl;
        cout << endl;

        cout << "THRUST TRANSPOSING:" << endl;
        init_zero_matrix(output_matrix);
        float elapsed_time_thrust_transpose = 0.0f;
                                elapsed_time_thrust_transpose             =
thrust_transpose(device_input_matrix_1, device_output_matrix);
          cudaMemcpy(output_matrix,  device_output_matrix,  NUM_ELEMENTS  *
sizeof(float), cudaMemcpyDeviceToHost);
        transpose_check(input_matrix_1, output_matrix);
        cout << "Time: " << elapsed_time_thrust_transpose << " ms" << endl;
        cout << endl;

        cout << "CUBLAS TRANSPOSING:" << endl;
        init_zero_matrix(output_matrix);
        float elapsed_time_cublas_transpose = 0.0f;
                                elapsed_time_cublas_transpose             =
cublas_transpose(device_input_matrix_1, device_output_matrix);
          cudaMemcpy(output_matrix,  device_output_matrix,  NUM_ELEMENTS  *
sizeof(float), cudaMemcpyDeviceToHost);
        transpose_check(input_matrix_1, output_matrix);
        cout << "Time: " << elapsed_time_cublas_transpose << " ms" << endl;
```

```
        cout << endl;

        cout << "KERNEL MATRIX MULTIPLICATION:" << endl;
        init_zero_matrix(output_matrix);
        float elapsed_time_kernel_matrix_multiplication = 0.0f;
                        elapsed_time_kernel_matrix_multiplication        =
kernel_matrix_multiply(device_input_matrix_1,        device_input_matrix_2,
device_output_matrix);
          cudaMemcpy(output_matrix,  device_output_matrix,  NUM_ELEMENTS  *
sizeof(float), cudaMemcpyDeviceToHost);
        multiply_check(input_matrix_1, input_matrix_2, output_matrix);
         cout << "Time: " << elapsed_time_kernel_matrix_multiplication << "
ms" << endl;
        cout << endl;

        cout << "THRUST MATRIX MULTIPLICATION:" << endl;
        init_zero_matrix(output_matrix);
        float elapsed_time_thrust_matrix_multiplication = 0.0f;
                        elapsed_time_thrust_matrix_multiplication        =
thrust_matrix_multiply(device_input_matrix_1,        device_input_matrix_2,
device_output_matrix);
          cudaMemcpy(output_matrix,  device_output_matrix,  NUM_ELEMENTS  *
sizeof(float), cudaMemcpyDeviceToHost);
        multiply_check(input_matrix_1, input_matrix_2, output_matrix);
         cout << "Time: " << elapsed_time_thrust_matrix_multiplication << "
ms" << endl;
        cout << endl;

        cout << "CUBLAS MATRIX MULTIPLICATION:" << endl;
        init_zero_matrix(output_matrix);
        float elapsed_time_cublas_matrix_multiplication = 0.0f;
                        elapsed_time_cublas_matrix_multiplication        =
cublas_matrix_multiply(device_input_matrix_1,        device_input_matrix_2,
device_output_matrix);
          cudaMemcpy(output_matrix,  device_output_matrix,  NUM_ELEMENTS  *
sizeof(float), cudaMemcpyDeviceToHost);
        multiply_check(input_matrix_1, input_matrix_2, output_matrix);
         cout << "Time: " << elapsed_time_cublas_matrix_multiplication << "
ms" << endl;
        cout << endl;

        cout << "KERNEL SCALAR MULTIPLICATION:" << endl;
        init_zero_matrix(output_matrix);
        float elapsed_time_kernel_scalar_multiplication = 0.0f;
                        elapsed_time_kernel_scalar_multiplication        =
kernel_scalar_multiply(device_input_matrix_1, device_output_matrix);
          cudaMemcpy(output_matrix,  device_output_matrix,  NUM_ELEMENTS  *
sizeof(float), cudaMemcpyDeviceToHost);
        scalar_check(input_matrix_1, output_matrix);
```

```
        cout << "Time: " << elapsed_time_kernel_scalar_multiplication << "
ms" << endl;
        cout << endl;

        cout << "THRUST SCALAR MULTIPLICATION:" << endl;
        init_zero_matrix(output_matrix);
        float elapsed_time_thrust_scalar_multiplication = 0.0f;
                    elapsed_time_thrust_scalar_multiplication         =
thrust_scalar_multiply(device_input_matrix_1, device_output_matrix);
          cudaMemcpy(output_matrix,  device_output_matrix,  NUM_ELEMENTS  *
sizeof(float), cudaMemcpyDeviceToHost);
        scalar_check(input_matrix_1, output_matrix);
        cout << "Time: " << elapsed_time_thrust_scalar_multiplication << "
ms" << endl;
        cout << endl;

        cout << "CUBLAS SCALAR MULTIPLICATION:" << endl;
        init_zero_matrix(output_matrix);
        float elapsed_time_cublas_scalar_multiplication = 0.0f;
                    elapsed_time_cublas_scalar_multiplication         =
cublas_scalar_multiply(device_input_matrix_1, device_output_matrix);
          cudaMemcpy(output_matrix,  device_output_matrix,  NUM_ELEMENTS  *
sizeof(float), cudaMemcpyDeviceToHost);
        scalar_check(input_matrix_1, output_matrix);
        cout << "Time: " << elapsed_time_cublas_scalar_multiplication << "
ms" << endl;
        cout << endl;

        cout << "TOTAL RESULTS:" << endl;
        cout << "MATRIX SIZE = " << MATRIX_SIZE << " x " << MATRIX_SIZE <<
" (" << NUM_ELEMENTS << " elements)" << endl;
        cout << setw(40) << "raw CUDA, ms" << setw(15) << "Thrust, ms"
             << setw(15) << "cuBLSAS, ms" << endl;
        cout << fixed;
          cout  <<  setw(25)  <<  "Matrix  transposing"  <<  setw(15)  <<
elapsed_time_kernel_transpose << setw(15) << elapsed_time_thrust_transpose <<
setw(15) << elapsed_time_cublas_transpose << endl;
          cout  <<  setw(25)  <<  "Matrix  multiplication"  <<  setw(15)  <<
elapsed_time_kernel_matrix_multiplication       <<       setw(15)       <<
elapsed_time_thrust_matrix_multiplication       <<       setw(15)       <<
elapsed_time_cublas_matrix_multiplication << endl;
          cout  <<  setw(25)  <<  "Scalar  multiplication"  <<  setw(15)  <<
elapsed_time_kernel_scalar_multiplication       <<       setw(15)       <<
elapsed_time_thrust_scalar_multiplication       <<       setw(15)       <<
elapsed_time_cublas_scalar_multiplication << endl;

        if (FILE_OUTPUT)
        {
            ofstream file;
            file.open("results.csv", ios::app);
```

```
            if (file.tellp() == streampos(0))
            {
                  file << "Matrix size (n^2); MT (CUDA); MT (Thrust); MT
(cuBLAS); MM (CUDA); MM (Thrust); MM (cuBLAS); SM (CUDA); SM (Thrust); SM
(cuBLAS);" << endl;
            }

            file << MATRIX_SIZE << ";"
                        <<  elapsed_time_kernel_transpose  <<  ";"  <<
elapsed_time_thrust_transpose << ";" << elapsed_time_cublas_transpose << ";"
                    << elapsed_time_kernel_matrix_multiplication << ";" <<
elapsed_time_thrust_matrix_multiplication        <<        ";"        <<
elapsed_time_cublas_matrix_multiplication << ";"
                    << elapsed_time_kernel_scalar_multiplication << ";" <<
elapsed_time_thrust_scalar_multiplication        <<        ";"        <<
elapsed_time_cublas_scalar_multiplication << ";"
                  << endl;

            file.close();
        }

        return EXIT_SUCCESS;
    }
```

matrix_handler.h

```
    #ifndef MATRIX_HANDLER
    #define MATRIX_HANDLER
    #include <iostream>
    #include "utility.h"

    #define GREEN "\033[32m"
    #define RED "\033[31m"
    #define STANDART_COLOR "\033[0m"

    using namespace std;

    void init_matrix(float *matrix)
    {
        srand(time(NULL));
        for (int i = 0; i < NUM_ELEMENTS; i++)
            matrix[i] = (float)(rand() % 90 + 10);
    }

    void init_zero_matrix(float *matrix)
    {
        for (int i = 0; i < NUM_ELEMENTS; i++)
            matrix[i] = 0.0f;
    }
```

```
void print_matrix(float *matrix)
{
    for (int i = 0; i < MATRIX_SIZE; i++)
    {
        for (int j = 0; j < MATRIX_SIZE; j++)
            cout << matrix[i * MATRIX_SIZE + j] << " ";
        cout << endl;
    }
    cout << endl;
}

#endif
```

cuda_handler.h

```
#ifndef CUDA_HANDLER
#define CUDA_HANDLER
#include <thrust/device_vector.h>
#include <thrust/iterator/zip_iterator.h>
#include <thrust/tuple.h>
#include <thrust/functional.h>
#include <thrust/transform.h>
#include <thrust/execution_policy.h>
#include <iostream>
#include <iomanip>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>
#include <cublas_v2.h>
#include "utility.h"
#include "matrix_handler.h"

using namespace std;

// TRANSPOSING PART

void transpose_check(float *input_matrix, float *transposed_matrix)
{
    bool is_equal = true;
       float   *expected_matrix   =   (float   *)malloc(NUM_ELEMENTS   *
sizeof(float));
    for (int i = 0; i < MATRIX_SIZE; i++)
        for (int j = 0; j < MATRIX_SIZE; j++)
            expected_matrix[j * MATRIX_SIZE + i] = input_matrix[i *
MATRIX_SIZE + j];

    if (PRINT_MODE)
    {
        cout << "INPUT MATRIX:" << endl;
```

```
            print_matrix(input_matrix);
            cout << "EXPECTED MATRIX:" << endl;
            print_matrix(expected_matrix);
            cout << "ACTUAL MATRIX:" << endl;
            print_matrix(transposed_matrix);
        }

        bool found_mismatch = false;
        for (int i = 0; i < MATRIX_SIZE; i++)
        {
            for (int j = 0; j < MATRIX_SIZE; j++)
                    if (transposed_matrix[i * MATRIX_SIZE + j] !=
expected_matrix[i * MATRIX_SIZE + j])
                {
                    is_equal = false;
                    found_mismatch = true;
                     cout << "Mismatch found at index (" << i << ", " << j
<< ")" << endl;
                    cout << "Expected: " << expected_matrix[i * MATRIX_SIZE
+ j] << endl;
                    cout << "Actual: " << transposed_matrix[i * MATRIX_SIZE
+ j] << endl;
                    break;
                }
            if (found_mismatch)
                break;
        }

        if (is_equal)
            cout << GREEN << "SUCCESS: Transposed matrix equal to expected"
<< STANDART_COLOR << endl;
        else
             cout << RED << "FAILURE: Transposed matrix doesn't equal to
expected" << STANDART_COLOR << endl;
    }

    __global__ void transpose_kernel(float *input_matrix, float
*output_matrix)
    {
        int x = blockIdx.x * blockDim.x + threadIdx.x;
        int y = blockIdx.y * blockDim.y + threadIdx.y;

        if (x < MATRIX_SIZE && y < MATRIX_SIZE)
        {
            int index_in = y * MATRIX_SIZE + x;
            int index_out = x * MATRIX_SIZE + y;
            output_matrix[index_out] = input_matrix[index_in];
        }
    }
```

```
    float kernel_transpose(float *input_matrix, float *output_matrix)
    {
        float elapsed_time = 0.0f;
        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);

        cudaEventRecord(start);

        dim3 dimGrid((MATRIX_SIZE - 1) / BLOCK_SIZE + 1, (MATRIX_SIZE - 1)
/ BLOCK_SIZE + 1);
        dim3 dimBlock(TILE_DIM, BLOCK_SIZE, 1);

                  transpose_kernel<<<dimGrid,     dimBlock>>>(input_matrix,
output_matrix);

        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&elapsed_time, start, stop);

        return elapsed_time;
    }

    float thrust_transpose(float *input_matrix, float *output_matrix)
    {
        float elapsed_time = 0.0f;
        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);

        cudaEventRecord(start);

        thrust::device_vector<float> input(NUM_ELEMENTS);
        thrust::device_vector<float> output(NUM_ELEMENTS);

        // Copy input data to input vector
            thrust::copy(input_matrix,   input_matrix   +   NUM_ELEMENTS,
input.begin());

        thrust::counting_iterator<int> first(0);
        thrust::counting_iterator<int> last(NUM_ELEMENTS);

        auto input_iter = thrust::make_permutation_iterator(input.begin(),
                                                            thrust::make_t
ransform_iterator(first,

                [=] __device__(int i) mutable

                {
```

```
                    int r = i / MATRIX_SIZE;

                    int c = i % MATRIX_SIZE;

                    return c * MATRIX_SIZE + r;

                }));

        thrust::copy(input_iter, input_iter + NUM_ELEMENTS, output.begin());

        // Copy output data back to output_matrix
        thrust::copy(output.begin(), output.end(), output_matrix);

        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&elapsed_time, start, stop);

        return elapsed_time;
    }

    float cublas_transpose(float *input_matrix, float *output_matrix)
    {
        float alpha = 1.0f;
        float beta = 0.0f;
        float elapsed_time = 0.0f;
        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);

        cublasHandle_t handle;
        cublasCreate(&handle);

        cudaEventRecord(start);
            cublasSgeam(handle, CUBLAS_OP_T, CUBLAS_OP_N, MATRIX_SIZE,
MATRIX_SIZE, &alpha, input_matrix, MATRIX_SIZE, &beta, input_matrix,
MATRIX_SIZE, output_matrix, MATRIX_SIZE);
        cudaEventRecord(stop);

        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&elapsed_time, start, stop);

        cublasDestroy(handle);

        return elapsed_time;
    }

    // MULTIPLYING PART
```

```
      void multiply_check(float *input_matrix_1, float *input_matrix_2, float
*result_matrix)
      {
          bool is_equal = true;
             float   *expected_matrix   =   (float   *)malloc(NUM_ELEMENTS   *
sizeof(float));
          for (int i = 0; i < MATRIX_SIZE; i++)
              for (int j = 0; j < MATRIX_SIZE; j++)
              {
                  float sum = 0.0f;
                  for (int k = 0; k < MATRIX_SIZE; k++)
                            sum  +=  input_matrix_1[i  *  MATRIX_SIZE  +  k]  *
input_matrix_2[k * MATRIX_SIZE + j];
                  expected_matrix[i * MATRIX_SIZE + j] = sum;
              }

          if (PRINT_MODE)
          {
              cout << "INPUT MATRIX 1:" << endl;
              print_matrix(input_matrix_1);
              cout << "INPUT MATRIX 2:" << endl;
              print_matrix(input_matrix_2);
              cout << "EXPECTED MATRIX:" << endl;
              print_matrix(expected_matrix);
              cout << "ACTUAL MATRIX:" << endl;
              print_matrix(result_matrix);
          }

          bool found_mismatch = false;
          for (int i = 0; i < MATRIX_SIZE; i++)
          {
              for (int j = 0; j < MATRIX_SIZE; j++)
                        if  (fabs(result_matrix[i  *  MATRIX_SIZE  +  j]  -
expected_matrix[i * MATRIX_SIZE + j]) > 1e-6f)
                  {
                      is_equal = false;
                      found_mismatch = true;
                       cout << "Mismatch found at index (" << i << ", " << j
<< ")" << endl;
                      cout << "Expected: " << expected_matrix[i * MATRIX_SIZE
+ j] << endl;
                       cout << "Actual: " << result_matrix[i * MATRIX_SIZE +
j] << endl;
                      break;
                  }
              if (found_mismatch)
                  break;
          }

          if (is_equal)
```

```
            cout << GREEN << "SUCCESS: Result matrix equal to expected" <<
STANDART_COLOR << endl;
        else
            cout << RED << "FAILURE: Result matrix doesn't equal to expected"
<< STANDART_COLOR << endl;
    }

    __global__ void matrix_multiply_kernel(float *input_matrix_1, float
*input_matrix_2, float *output_matrix)
    {
        int row = blockIdx.y * blockDim.y + threadIdx.y;
        int col = blockIdx.x * blockDim.x + threadIdx.x;

        if (row < MATRIX_SIZE && col < MATRIX_SIZE)
        {
            float sum = 0.0f;
            for (int i = 0; i < MATRIX_SIZE; i++)
            {
                    sum += input_matrix_1[row * MATRIX_SIZE + i] *
input_matrix_2[i * MATRIX_SIZE + col];
            }
            output_matrix[row * MATRIX_SIZE + col] = sum;
        }
    }

    float kernel_matrix_multiply(float *input_matrix_1, float
*input_matrix_2, float *output_matrix)
    {
        float elapsed_time = 0.0f;
        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);

        cudaEventRecord(start);

        dim3 dimGrid((MATRIX_SIZE - 1) / BLOCK_SIZE + 1, (MATRIX_SIZE - 1)
/ BLOCK_SIZE + 1);
        dim3 dimBlock(TILE_DIM, BLOCK_SIZE, 1);

            matrix_multiply_kernel<<<dimGrid, dimBlock>>>(input_matrix_1,
input_matrix_2, output_matrix);

        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&elapsed_time, start, stop);

        return elapsed_time;
    }
```

```
    float     thrust_matrix_multiply(float     *input_matrix_1,     float
*input_matrix_2, float *output_matrix)
    {
        float elapsed_time = 0.0f;
        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);

        cudaEventRecord(start);

        thrust::device_ptr<float> d_input_matrix_1(input_matrix_1);
        thrust::device_ptr<float> d_input_matrix_2(input_matrix_2);
        thrust::device_ptr<float> d_output_matrix(output_matrix);

        thrust::counting_iterator<int> idx_first(0);
        thrust::counting_iterator<int> idx_last = idx_first + MATRIX_SIZE *
MATRIX_SIZE;

        thrust::transform(
            idx_first, idx_last,
            d_output_matrix,
           [d_input_matrix_1, d_input_matrix_2] __device__(int idx) -> float
            {
                int row = idx / MATRIX_SIZE;
                int col = idx % MATRIX_SIZE;
                float sum = 0.0f;
                for (int i = 0; i < MATRIX_SIZE; i++)
                {
                        sum += d_input_matrix_1[row * MATRIX_SIZE + i] *
d_input_matrix_2[i * MATRIX_SIZE + col];
                }
                return sum;
            });

        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&elapsed_time, start, stop);

        return elapsed_time;
    }

    float     cublas_matrix_multiply(float     *input_matrix_1,     float
*input_matrix_2, float *output_matrix)
    {
        float alpha = 1.0f;
        float beta = 0.0f;

        cublasHandle_t handle;
        cublasCreate(&handle);
```

```
            cudaEvent_t start, stop;
            cudaEventCreate(&start);
            cudaEventCreate(&stop);

            cudaEventRecord(start);

                cublasSgemm(handle,   CUBLAS_OP_N,   CUBLAS_OP_N,   MATRIX_SIZE,
MATRIX_SIZE,     MATRIX_SIZE,     &alpha,     input_matrix_2,     MATRIX_SIZE,
input_matrix_1, MATRIX_SIZE, &beta, output_matrix, MATRIX_SIZE);

            cudaEventRecord(stop);
            cudaEventSynchronize(stop);
            float elapsed_time;
            cudaEventElapsedTime(&elapsed_time, start, stop);

            cublasDestroy(handle);

            return elapsed_time;
        }

        /*Scalar multiplication of matrices is the operation of multiplying each
element of a matrix by a scalar value (a single number).
        In other words, given a matrix A and a scalar k, the scalar multiplication
of A and k is a new matrix C
        where each element of A is multiplied by k to get the corresponding
element of C. For example, if A = [1 2; 3 4] and k = 2,
        then the scalar multiplication of A and k is C = [2 4; 6 8].
        A scalar is a unit matrix with a necessary number on the main diagonal.
        For example, if scalar k = 2 and width of matrix A is 2, then k => k *
E_2 => [2 0; 0 2]*/

        void scalar_check(float *input_matrix, float *result_matrix)
        {
            bool is_equal = true;
                float   *expected_matrix   =   (float   *)malloc(NUM_ELEMENTS   *
sizeof(float));
            for (int i = 0; i < NUM_ELEMENTS; i++)
                expected_matrix[i] = SCALAR * input_matrix[i];

            if (PRINT_MODE)
            {
                cout << "INPUT MATRIX:" << endl;
                print_matrix(input_matrix);
                cout << "SCALAR = " << SCALAR << endl;
                cout << "EXPECTED MATRIX:" << endl;
                print_matrix(expected_matrix);
                cout << "ACTUAL MATRIX:" << endl;
                print_matrix(result_matrix);
            }
```

```
        bool found_mismatch = false;
        for (int i = 0; i < MATRIX_SIZE; i++)
        {
            for (int j = 0; j < MATRIX_SIZE; j++)
                if (result_matrix[i * MATRIX_SIZE + j] != expected_matrix[i
* MATRIX_SIZE + j])
                {
                    is_equal = false;
                    found_mismatch = true;
                     cout << "Mismatch found at index (" << i << ", " << j
<< ")" << endl;
                    cout << "Expected: " << expected_matrix[i * MATRIX_SIZE
+ j] << endl;
                     cout << "Actual: " << result_matrix[i * MATRIX_SIZE +
j] << endl;
                    break;
                }
            if (found_mismatch)
                break;
        }

        if (is_equal)
             cout << GREEN << "SUCCESS: Result matrix equal to expected" <<
STANDART_COLOR << endl;
        else
            cout << RED << "FAILURE: Result matrix doesn't equal to expected"
<< STANDART_COLOR << endl;
    }

    __global__ void scalar_multiply_kernel(float *input_matrix, float
*output_matrix)
    {
        int x = blockIdx.x * blockDim.x + threadIdx.x;
        int y = blockIdx.y * blockDim.y + threadIdx.y;

        if (x < MATRIX_SIZE && y < MATRIX_SIZE)
            output_matrix[y * MATRIX_SIZE + x] = input_matrix[y * MATRIX_SIZE
+ x] * SCALAR;
    }

    float kernel_scalar_multiply(float *input_matrix, float *output_matrix)
    {
        float elapsed_time = 0.0f;
        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);

        cudaEventRecord(start);
```

```
        dim3 dimGrid((MATRIX_SIZE - 1) / BLOCK_SIZE + 1, (MATRIX_SIZE - 1)
/ BLOCK_SIZE + 1);
        dim3 dimBlock(TILE_DIM, BLOCK_SIZE, 1);

             scalar_multiply_kernel<<<dimGrid,    dimBlock>>>(input_matrix,
output_matrix);

        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&elapsed_time, start, stop);

        return elapsed_time;
    }

    float thrust_scalar_multiply(float *input_matrix, float *output_matrix)
    {
        float elapsed_time = 0.0f;
        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);

        cudaEventRecord(start);

        thrust::device_ptr<float> d_input_matrix(input_matrix);
        thrust::device_ptr<float> d_output_matrix(output_matrix);

        thrust::counting_iterator<int> idx_first(0);
        thrust::counting_iterator<int> idx_last = idx_first + MATRIX_SIZE *
MATRIX_SIZE;

        thrust::transform(
            idx_first, idx_last,
            d_output_matrix,
            [d_input_matrix] __device__(int idx) -> float
            {
                return d_input_matrix[idx] * SCALAR;
            });

        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&elapsed_time, start, stop);

        return elapsed_time;
    }

    float cublas_scalar_multiply(float *input_matrix, float *output_matrix)
    {
        cublasHandle_t handle;
        cublasCreate(&handle);
```

```
        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);

        cudaEventRecord(start);

        float scalar = static_cast<float>(SCALAR);
            cublasSscal(handle,   MATRIX_SIZE   *   MATRIX_SIZE,   &scalar,
input_matrix, 1);

        cudaMemcpy(output_matrix, input_matrix, MATRIX_SIZE * MATRIX_SIZE *
sizeof(float), cudaMemcpyDeviceToDevice);

        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        float elapsed_time;
        cudaEventElapsedTime(&elapsed_time, start, stop);

        cublasDestroy(handle);

        return elapsed_time;
    }

    #endif
```

utility.h

```
    #ifndef UTILITY
    #define UTILITY

    #define MATRIX_SIZE 512
    #define NUM_ELEMENTS (MATRIX_SIZE * MATRIX_SIZE)
    #define BLOCK_SIZE 16 // programm crashes if > 22
    #define TILE_DIM (BLOCK_SIZE * 2)
    #define SCALAR 10

    #define PRINT_MODE false  // change between true/false
    #define FILE_OUTPUT false // change between true/false

    #endif
```